

BTS Services Informatiques aux Organisations (SIO)

Session 2026

Nom du candidat : SEILLER

Prénom du candidat : Julian

Parcours : SLAM

Fiche de réalisation professionnelle N°9

Développement d'une application de gestion hôtelière (Projet Hotel California)

Période de réalisation : 1ère et 2ème année de BTS

Type de réalisation : En formation (Lycée Louise Michel)

Mode de réalisation : En équipe (Binôme)

Sommaire

Contexte Organisationnel.....	3
Problématique de gestion.....	3
Choix de la solution mise en œuvre.....	3
Réalisation de la solution.....	4
a - Prérequis et ressources à disposition.....	4
b - Mise en œuvre de la solution.....	4
c - Tests.....	4
i - Cas d'utilisation 1 : Affichage et rendu dynamique (EJS)	4
ii - Cas d'utilisation 2 : Validation des données entrantes.	5
iii - Cas d'utilisation 3 : Opération CRUD complète.....	5
Conclusions.....	5
Annexes.....	5
Compétences associées.....	6

Contexte Organisationnel

Cette réalisation a été effectuée dans le cadre de ma formation au **Lycée Louise Michel**, en cours de **SLAM**. Le projet consiste à répondre aux besoins d'informatisation d'un établissement hôtelier fictif nommé "**Hotel California**". L'objectif est de fournir au personnel un outil Web centralisé, performant et sécurisé pour gérer les chambres, les clients et les réservations.

Problématique de gestion

L'établissement souffrait d'une gestion décentralisée provoquant des **surbookings** et des lenteurs administratives. Il fallait concevoir une application Web capable de traiter rapidement des requêtes simultanées, tout en garantissant la sécurité des données saisies (prévention des injections, vérification des formulaires). De plus, le code source devait être **rigoureusement structuré** pour permettre à plusieurs étudiants de travailler dessus en équipe sans générer de conflits.

Choix de la solution mise en œuvre

Pour répondre à ce besoin de performance et de modularité, nous avons opté pour une stack technique **JavaScript Full-Stack** :

- **Back-end** : Environnement d'exécution **Node.js** couplé au framework **Express.js**, idéal pour construire un serveur Web léger et rapide.
- **Architecture MVC** : Séparation stricte du code en dossiers spécialisés (models, views, controllers, routes) pour garantir la maintenabilité.
- **Moteur de template** : Utilisation de **EJS (Embedded JavaScript)** pour générer dynamiquement le code HTML côté serveur avant de l'envoyer au navigateur.
- **Sécurité & Contrôle** : Création de dossiers dédiés pour les **middleware** (vérification des sessions), les **validators** (contrôle des saisies utilisateurs), et configuration **ssl** pour le HTTPS.

Réalisation de la solution

a - Prérequis et ressources à disposition

- Environnement **Node.js** installé avec le gestionnaire de paquets **NPM**.
- Éditeur de code (VS Code).
- Fichier **.env** pour isoler les variables d'environnement (identifiants de base de données, ports de connexion) et **.gitignore** pour sécuriser le dépôt.

b - Mise en œuvre de la solution

- **Configuration du serveur et Routage** : J'ai configuré le serveur Express et défini les points de terminaison dans le dossier routes. Ces routes redirigent les requêtes HTTP vers les fonctions appropriées situées dans le dossier controllers.
- **Développement des Vues (EJS)** : J'ai conçu les interfaces utilisateur dans le dossier views. J'ai créé des fichiers spécifiques pour chaque action CRUD : index.ejs (liste), showOne.ejs (détails), create.ejs (formulaire d'ajout), edit.ejs (modification) et delete.ejs (confirmation de suppression).
- **Validation et Middlewares** : Pour sécuriser l'application, j'ai codé des scripts dans le dossier validators afin de vérifier que les données des formulaires sont conformes avant de solliciter les models (base de données). Les middlewares ont été utilisés pour bloquer l'accès aux routes privées si l'utilisateur n'est pas authentifié.

c - Tests

i - Cas d'utilisation 1 : Affichage et rendu dynamique (EJS)

- **Entrée / Action** : L'utilisateur accède à la route /reservations pour voir la liste des séjours.
- **Résultat attendu** : Le contrôleur interroge la base de données, récupère un tableau d'objets, et le transmet à la vue index.ejs. La page Web s'affiche avec la liste complète générée dynamiquement.

ii - Cas d'utilisation 2 : Validation des données entrantes

- **Entrée / Action** : Soumission du formulaire create.ejs avec une date de départ antérieure à la date d'arrivée.
- **Résultat attendu** : Le script situé dans le dossier validators intercepte la requête HTTP, bloque l'insertion en base de données, et renvoie un message d'erreur clair à l'utilisateur.

iii - Cas d'utilisation 3 : Opération CRUD complète

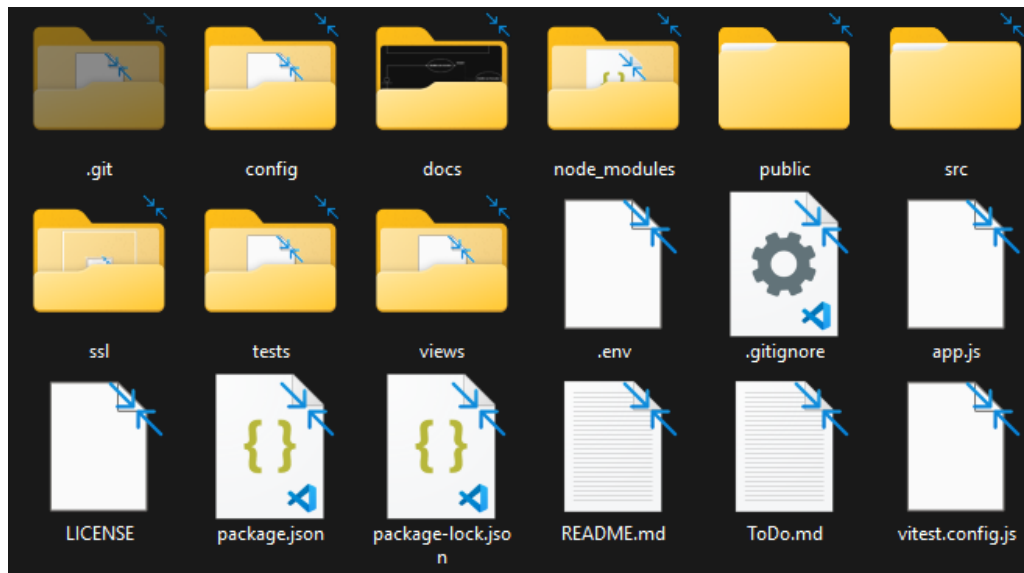
- **Entrée / Action** : Clic sur le bouton de suppression d'une chambre depuis l'interface, ce qui charge la vue delete.ejs pour confirmation.
- **Résultat attendu** : Après validation, la requête HTTP DELETE est envoyée. Le contrôleur supprime l'entrée via le modèle, puis redirige l'utilisateur vers index.ejs mis à jour.

Conclusions

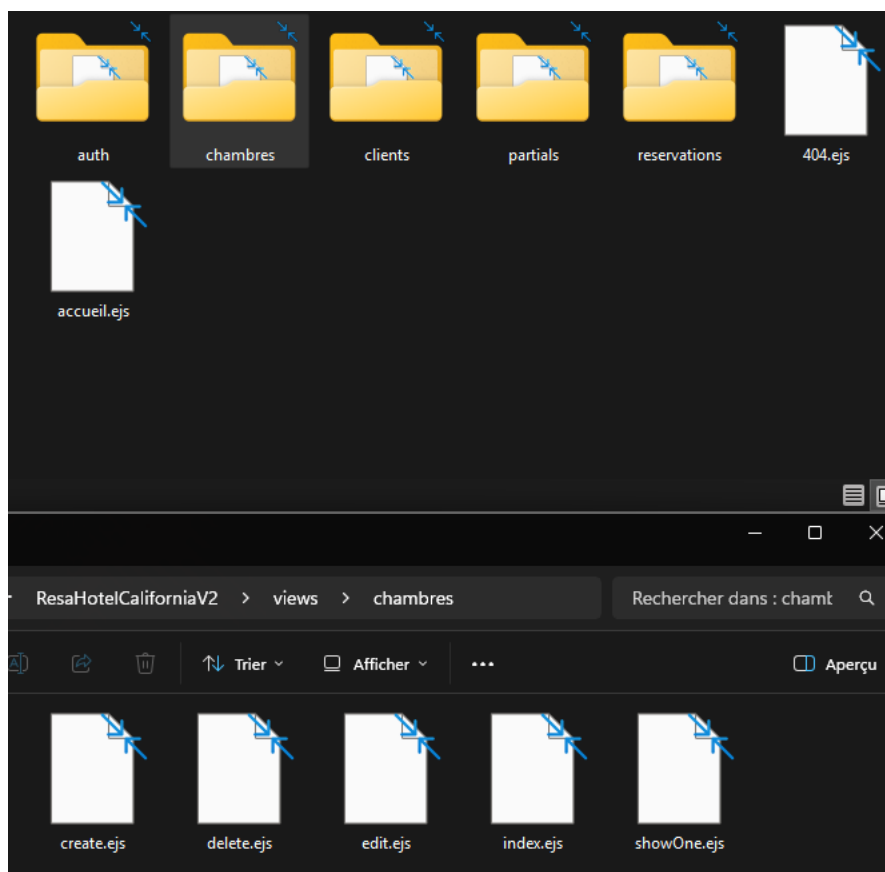
Ce projet m'a permis de maîtriser l'écosystème **Node.js / Express**, très prisé dans le monde professionnel. J'ai appris l'importance d'une architecture modulaire (séparation des routes, contrôleurs et vues) pour garder un code lisible en équipe. La mise en place de **middlewares** et de **validateurs** m'a également sensibilisé aux problématiques de sécurité essentielles dans le développement d'applications Web.

Annexes

- **Annexe 1 :** Capture de l'arborescence du projet (dossiers src, controllers, models, validators, etc.).



- **Annexe 2 :** Capture du dossier views montrant les fichiers d'interface .ejs (create.ejs, edit.ejs, etc.).



Compétences associées

N°	Compétences associées
B2.1.4	Concevoir et développer une solution applicative (Node.js / Express)
B1.2.2	Assurer la maintenance évolutive (Architecture MVC modulaire)
B3.5.3b	Protéger les données à caractère personnel (Middlewares et Validators)
B1.4.2	Travailler en équipe informatique (Partage de config via Git)